Procedurális textúragenerálás OpenCL és OpenGL segítségével

Párhuzamos eszközök programozása – Beadandó feladat

1. Bevezetés

A projekt célja a procedurális textúragenerálás bemutatása OpenCL környezetben, különös tekintettel a Perlin-zaj algoritmus párhuzamosítására. A procedurális textúrák, mint például a felhők, füst vagy márvány mintázatok, matematikailag jól definiáltak, de számításigényesek, mivel minden egyes képpont értékét egyedi hash-függvények és interpolációk alapján kell meghatározni. Ez a probléma természetéből adódóan "embarrassingly parallel", így ideális jelölt a GPU-n történő végrehajtásra.

2. Fejlesztési folyamat

2.1. Első fázis: 2D NDRange és memória-leképezés verifikációja

A fejlesztés első szakaszában nem a zajgeneráláson, hanem az OpenCL infrastruktúra és a kétdimenziós munkatartomány (**NDRange**) helyes beállításán volt a hangsúly.

Mivel az OpenCL a memóriát egydimenziós pufferként kezeli, de a képalkotás kétdimenziós koordináta-rendszerben történik, elengedhetetlen a globális azonosítók (**global ID**) és a memóriacímek közötti pontos leképezés.

Megvalósítás: Ebben a fázisban egy 4×4-es tesztrácsot hoztam létre. A kernel feladata mindössze annyi volt, hogy a `get_global_id(0)` (oszlop) és `get_global_id(1)` (sor) értékekből egy jól azonosítható lebegőpontos értéket számoljon:

```
output[y * width + x] = (float)x + (y * 10.0f);
```

Ez a módszer lehetővé tette a konzolos kimeneten keresztül annak ellenőrzését, hogy:

1. A **Host** kód sikeresen inicializálta-e a **Device**-ot.
2. A 2D-s munkatartomány megfelelően lett-e kiosztva.
3. A kernel helyes indexszel írja-e a globális memóriát.

Eredmény:

A konzolon megjelenő 4×4-es mátrix igazolta, hogy a memória-címzés (offset) számítása helyes, így az alaprendszer készen áll a komplexebb matematikai algoritmusok fogadására.

```
# 4x4 OpenCL Grid Verification:
  0.0   1.0   2.0   3.0
 10.0  11.0  12.0  13.0
 20.0  21.0  22.0  23.0
 30.0  31.0  32.0  33.0
```

2.2. Második fázis: Pszeudo-véletlen zaj és rács-műtermékek

A zajgenerálás alapja a determinisztikus véletlenszerűség. Egy olyan függvényre van szükségünk, amely ugyanarra a bemeneti koordinátára mindig ugyanazt a kimenetet adja, de a szomszédos pixelek között nincs látható összefüggés.

Ebben a szakaszban egy klasszikus, szinusz alapú hash-függvényt alkalmaztam:

$$f(x, y) = \text{frac}(\sin(x \cdot 12.9898 + y \cdot 78.233) \cdot 43758.5453)$$

ahol a $\text{frac}(x)$ a szám törtrészét jelöli: $\text{frac}(x) = x - \lfloor x \rfloor$.

Tapasztalatok: Bár az algoritmus technikailag működik, az eredmény vizuálisan nem megfelelő:

1. **Lattice Artifacts:** A zaj túlságosan "éles", és nagyobb felbontásnál szabályos, ismétlődő mintázatok (moaré-hatás) figyelhetők meg. Ez a hash-függvény alacsony entrópiájából és a lebegőpontos precíziós hibákból adódik.
2. **Interpoláció hiánya:** Mivel minden pixel független a szomszédaitól, nem alakulnak ki összefüggő alakzatok, az eredmény inkább a televíziók "adásszünet" zajára emlékeztet.

Ez a fázis rávilágított arra, hogy a Perlin-zajhoz nem pixelenkénti véletlenszámokra, hanem egy rács-alapú gradiens-rendszerre és megfelelő interpolációra van szükség.

Miért pont ezek a számok?

A számítógépes grafikában (különösen a shaderek világában) a cél egy olyan függvény, ami:

1. **Determinisztikus:** Ugyanarra az (x, y) koordinátára mindig ugyanazt az értéket adja.
2. **Kaotikus:** Két egymáshoz nagyon közeli pont (pl. két szomszédos pixel) esetén a kimenet drasztikusan térjen el.

A képlet három részből áll:

1. **A skaláris szorzat (dot product):**

   A $x \cdot 12.9898 + y \cdot 78.233$ rész célja, hogy minden (x, y) párhoz egy egyedi, de fix számot rendeljen. Ezek a számok (12.9898 és 78.233) irracionális számok vagy prímek közelítései. Azért választottuk őket, hogy az x és y tengelyek mentén ne alakuljon ki könnyen ismétlődő mintázat.

2. **A szinusz függvény:**

   A $\sin(\dots)$ függvény gondoskodik a nemlinearitásról. A szinusz értéke -1 és 1 között mozog. Ha egy nagy számot adunk át a szinusznak, a kimenete nagyon érzékennyé válik a bemenet apró változásaira is.

3. **A nagy szorzó (43758.5453):**

   A szinusz értékét (ami -1 és 1 között van) megszorozzuk egy hatalmas számmal. Ezzel "szétnyújtjuk" a szinuszhullámot olyan vékonyra, hogy a lebegőpontos számábrázolás (float) pontossági korlátai miatt a tizedesjegyek teljesen kaotikussá válnak. Amikor a `fract()` függvénnyel levágjuk az egész részt, csak ez a kaotikus "zaj" marad meg.

Mi a baj velük?

- **Periodicitás:** Mivel a szinusz periodikus függvény (2π szerint), egy bizonyos távolság után a mintázat elkerülhetetlenül ismétlődni fog. Ezt nevezzük **aliasing**-nak vagy moaré-hatásnak.
- **Hardverfüggőség:** A 43758.5453-as szorzó a 24 bites mantisszával rendelkező lebegőpontos számokra van optimalizálva (standard IEEE 754 float). Különböző videókártyák (pl. egy régebbi mobil GPU vs. egy modern NVIDIA kártya) máshogy kerekíthetik a szinusz értékét, így ugyanaz a kód máshogy nézhet ki különböző eszközökön.

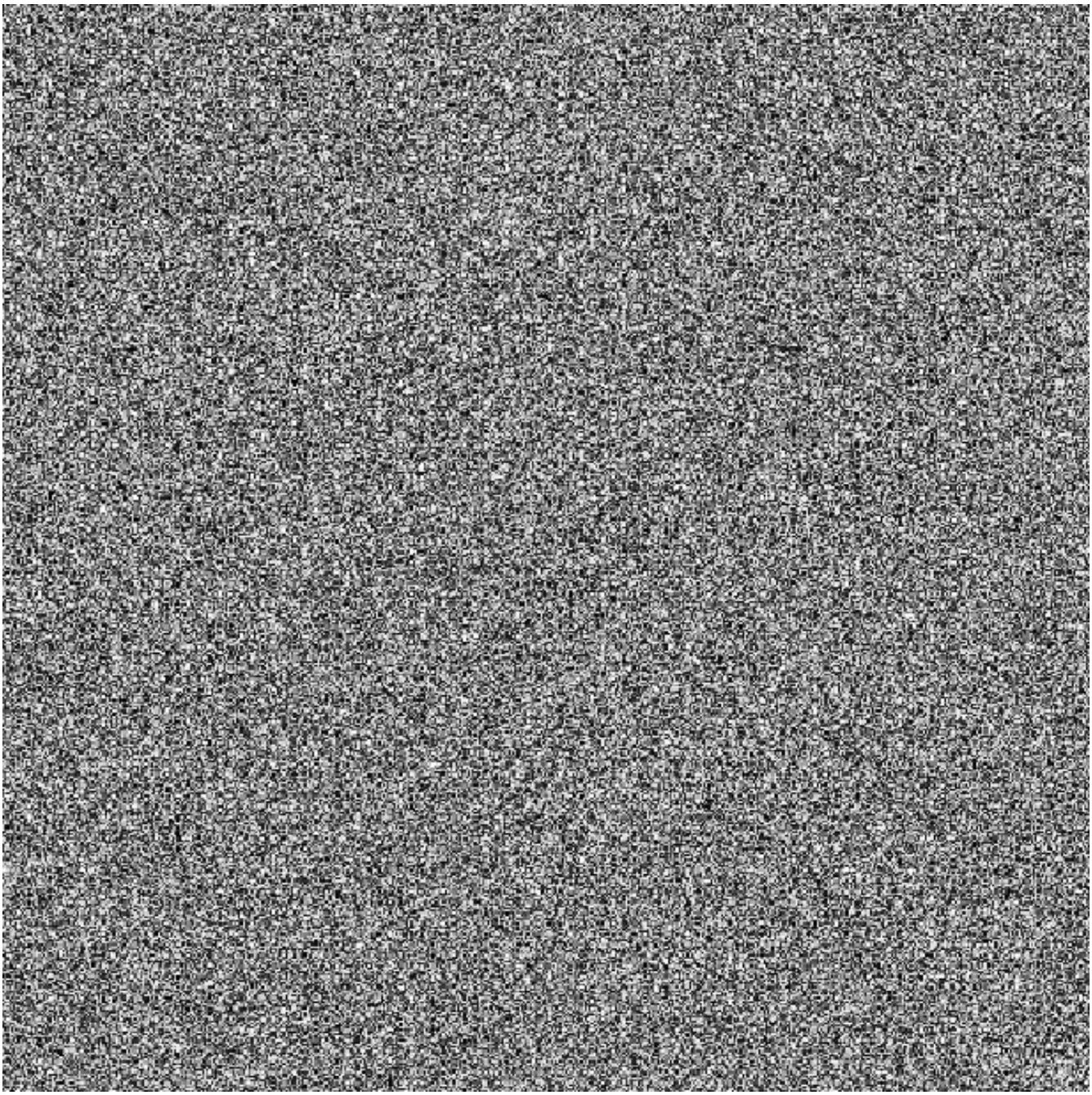

Figure 1: Basic Implementation

2.3. Harmadik fázis: Koherens zaj és gradiens rácsok

A Perlin-zaj lényege, hogy nem véletlen értékeket, hanem véletlen **irányvektorokat** (gradienseket) rendelünk egy rácsháló pontjaihoz. Egy adott pixel értékét a környező rácspontok gradiensei és a pixeltől való távolságuk skaláris szorzata határozza meg.

Az algoritmus matematikai lépései:

1. **Gradiens kijelölés:** Minden rácspontban egy pszeudo-véletlen egységvektort határozunk meg.
2. **Skaláris szorzat:** Kiszámítjuk a pixel és a négy legközelebbi rácspont közötti távolságvektor, valamint a rácsponti gradiens skaláris szorzatát.
3. **Simítás (Fade):** A lineáris interpoláció vizuálisan éles töréseket okozna a rácshatárnál. Ken Perlin javaslata alapján egy ötödfokú Hermite-interpolációs görbét használunk:

   $$S(t) = 6t^5 - 15t^4 + 10t^3$$

   Ez a függvény biztosítja, hogy a zaj első és másodrendű deriváltjai is folytonosak legyenek, ami megakadályozza a látható rácsvonalak kialakulását.
4. **Interpoláció:** A kapott értékeket bilineáris interpolációval egyesítjük.

Tapasztalatok: A korábbi fázissal ellentétben itt már összefüggő, organikus formák alakultak ki. A zaj skálázhatóvá vált a `scale` paraméter segítségével. Ez a megvalósítás képezi az alapját minden további textúragenerálásnak, azonban vizuálisan még mindig túl "egyszerűnek" hat a természetes jelenségekhez (pl. felhőkhöz) képest.

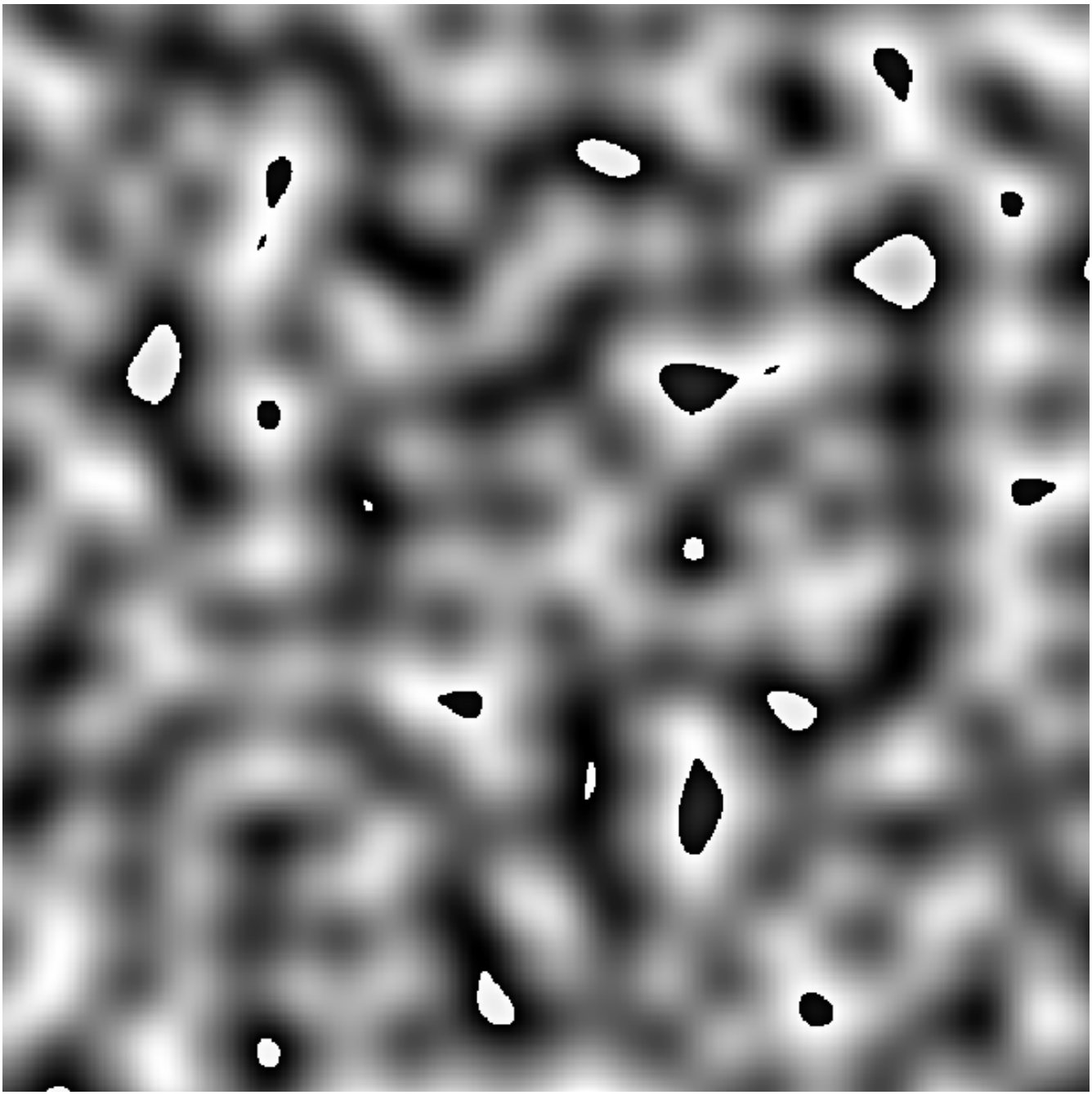

Figure 2: Quintic Interpolation

2.4. Negyedik fázis: Fraktális Brown-mozgás (fBm)

A természetes textúrák önhasonlóak: a nagyobb formák mellett kisebb, finomabb részletek is megfigyelhetők. Ennek szimulálására a Fraktális Brown-mozgás (fBm) technikát alkalmaztam.

Működési elv: Az fBm során több zaj-réteget (úgynevezett oktávot) adunk össze. Minden egyes iteráció során:

1. Növeljük a **frekvenciát** (jellemzően duplázzuk), így a zaj részletgazdagabb lesz.
2. Csökkentjük az **amplitúdót** (jellemzően felezzük), így a részletek kevésbé dominálnak, mint a fő forma.

A számítás matematikai képlete:

$$f(x, y) = \sum_{i=0}^{n-1} \text{noise}(2^i \cdot x, 2^i \cdot y) \cdot 0.5^i$$

ahol (n) az oktávok száma. Az én megvalósításomban ($n = 6$) réteget használtam.

Párhuzamosítási előnyök: Bár a ciklus bevezetése növelte a pixelenkénti számítási igényt, az OpenCL-nek köszönhetően ez alig érezhető a futási időben. A GPU-n futó több ezer szál párhuzamosan végzi el ezt a 6 iterációt minden egyes képpontra, ami a CPU-hoz képest nagyságrendekkel gyorsabb képalkotást tesz lehetővé.

Tapasztalatok: Az eredmény vizuálisan drasztikusan javult: a korábbi egyszerű foltok helyett megjelentek a felhőkre vagy füstre emlékeztető finom részletek. Ez a textúra már alkalmas professzionális felhasználásra is.

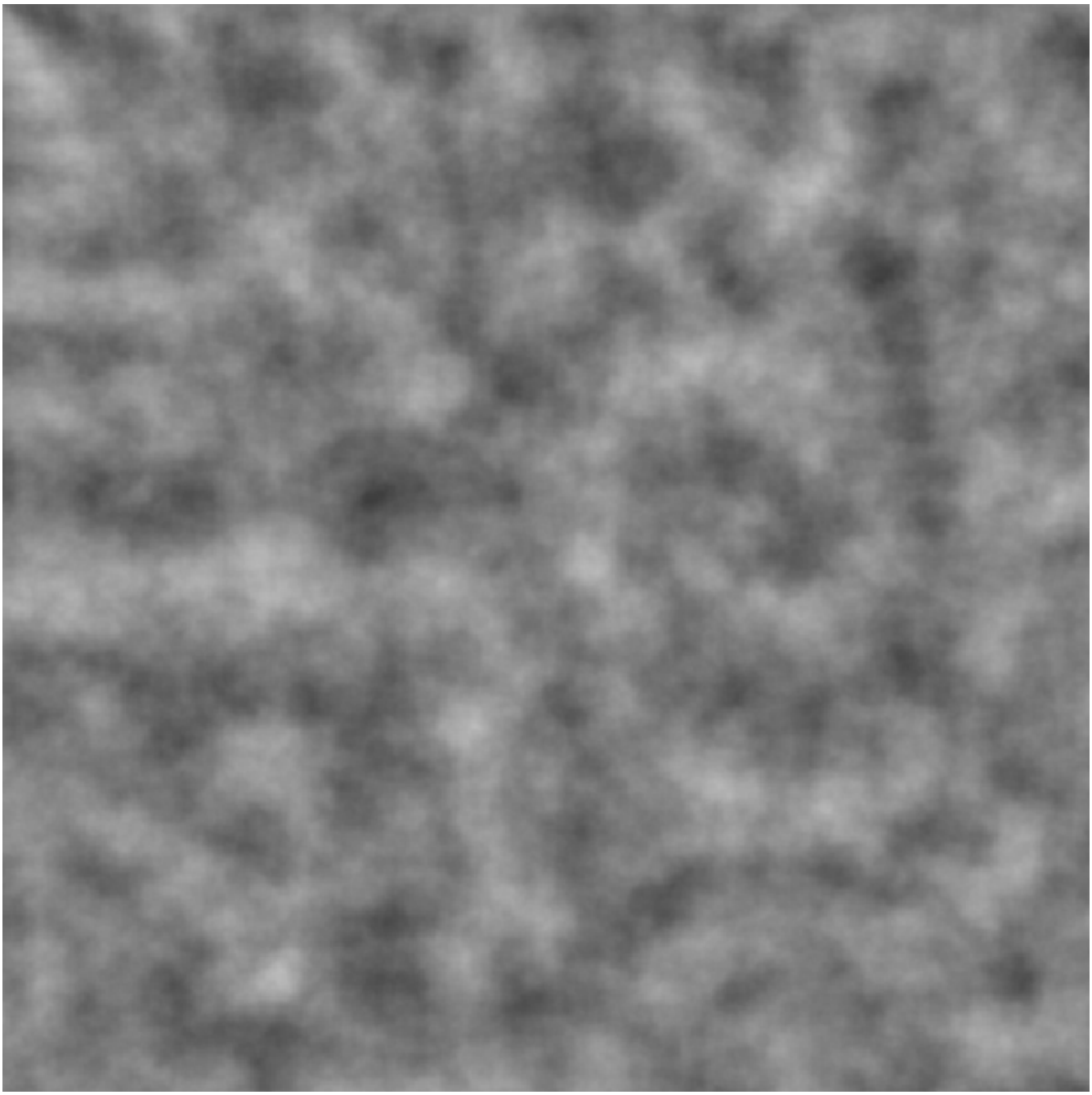

Figure 3: Fractal Brownian Motion

3. Teljesítményértékelés és Skálázódásvizsgálat

A párhuzamosítás hatékonyságának és az OpenCL work-group méretek hardveres hatásának vizsgálatára egy átfogó mérési sorozatot végeztem. A tesztek során a felbontást folyamatosan növeltem (960x540-től egészen 4K UHD-ig, 3840x2160-ig), és négy különböző végrehajtási stratégiát hasonlítottam össze:

- T_1 **(Szekvenciális CPU):** Referencia implementáció natív C nyelven, egyetlen processzorszálon futtatva.
- T_2 **(GPU, $\{1,1\}$ Local Work Size):** OpenCL végrehajtás, ahol a lokális munkacsoport mérete pixelenként 1x1.
- T_3 **(GPU, $\{16,16\}$ Local Work Size):** OpenCL végrehajtás optimalizált, 256 szálat (16x16) tartalmazó munkacsoportokkal.
- T_4 **(GPU, `NULL` Local Work Size):** OpenCL végrehajtás, ahol a driver automatikusan határozza meg az optimális munkacsoport-méretet.

3.1. CPU vs. GPU alapvető különbségek (T_1 vs. többi) Az eredmények drasztikus különbséget mutatnak a szekvenciális és a párhuzamosított végrehajtás között. A CPU (T_1) már a legkisebb, qHD (960x540) felbontáson is $\sim$ 738 ms alatt renderelt le egy képkockát, ami nem éri el a 2 FPS-t sem. A felbontás 4K-ra növelésével a CPU-s számítási idő lineárisan skálázódott a pixelek számával, így a futási idő elérte a **12 másodpercet** (12 002 ms) képkockánként. Ez bizonyítja, hogy a procedurális Perlin-noise valós idejű megjelenítésre CPU-n alkalmatlan.

Ezzel szemben a GPU még a legrosszabbul konfigurált esetben (T_2) is nagyságrendekkel gyorsabb volt, az optimalizált (T_3) esetben pedig 4K felbontás mellett is 1 **ms alatti** (0.719 ms) időt produkált.

3.2. A Local Work Size hatása a GPU-n

1. **A hardver alulkihasználtsága (T_2):** Az $\{1,1\}$ méretű munkacsoport katasztrofális a GPU architektúrájára nézve. A modern GPU-k (NVIDIA és AMD) fix méretű csoportokban (Warp: 32 szál, vagy Wavefront: 64 szál) ütemezik a végrehajtást. Ha a munkacsoport mérete $1 \times 1 = 1$, a GPU kénytelen elindítani egy teljes 32-es warpot, amiből 31 szál tétlenül áll (idle). Bár a T_2 még így is $\sim$ 500-szor gyorsabb a CPU-nál, a hardver kapacitásának csupán töredékét használja.
2. **Az optimális blokkméret (T_3):** A manuálisan megadott $\{16,16\}$ méret $16 \times 16 = 256$ szálat jelent munkacsoportonként. Ez tökéletes többszöröse a GPU warp méretének, maximális "occupancy"-t (szál-kihasználtságot) eredményezve a Compute Unit-okon (CU). Ez a konfiguráció adta a leggyorsabb futási időket általagosan $\sim$ 18800**-szoros gyorsulást** elérve a CPU-hoz képest.
3. **Driver általi optimalizáció (T_4):** Amikor a lokális méretet `NULL`-ra állítottam, az OpenCL driver heurisztikája próbálta megtalálni a legjobb felosztást. Bár a driver kiváló munkát végzett (átlagosan $\sim$ 12400-szoros gyorsulás), látható, hogy az általános célú beépített algoritmus $\sim 20-40\%$-kal elmaradt a manuálisan hangolt $\{16,16\}$-os méret teljesítményétől (pl. 4K esetén 0.763 ms a 0.719 ms-mal szemben).

3.3. Konklúzió A mérések egyértelműen igazolják a heterogén számítási modell (OpenCL) létjogosultságát grafikai algoritmusoknál. Továbbá rávilágítanak arra is, hogy az OpenCL kód

puszta megírása nem elegendő a maximális teljesítményhez: a GPU fizikai felépítéséhez igazodó, kézzel optimalizált munkacsoport-kiosztás (T_3) elengedhetetlen a hardverben rejlő potenciál teljes kiaknázásához.

4. Ötödik fázis: Valós idejű animáció és OpenGL Interoperabilitás

A projekt végső szakasza a statikus képfájlok (`.ppm`) generálásának leváltása egy valós idejű, interaktív grafikus alkalmazásra. A hagyományos megközelítés – miszerint az OpenCL kiszámolja az adatokat, azokat visszamásoljuk a rendszermemóriába (RAM), majd feltöltjük az OpenGL textúrájába – a PCIe sávszélesség korlátai miatt jelentős teljesítményveszteséggel járna.

Technikai megvalósítás (Zero-copy): Ennek elkerülésére az **OpenCL-OpenGL Interoperabilitást** (Shared Context) alkalmaztam a `cl_khr_gl_sharing` kiterjesztés segítségével.

1. A Host kód átadja a Windows-specifikus eszközleírókat (`HDC`, `HGLRC`) az OpenCL kontextusnak.
2. Az OpenGL-ben létrehozott textúrát memóriahivatkozásként (`image2d_t`) adjuk át a kernelnek.
3. A rajzolási ciklusban az OpenCL a `clEnqueueAcquireGLObjects` hívással "lefoglalja" a textúrát, elvégzi a párhuzamos zajgenerálást, majd a `clEnqueueReleaseGLObjects` hívással visszaadja az OpenGL-nek kirajzolásra.
4. Az adatok így soha nem hagyják el a GPU VRAM-ját (zero-copy), ami drasztikusan növeli a sebességet.

Interaktív "Végtelen" Terepgenerálás (Top-Down Map): A vizualizációt egy felülnézeti térképpé fejlesztettem tovább. A kernel a normalizált $[0, 1]$ közötti magasságértékeket egyszerű küszöbértékek (thresholds) alapján "biomokhoz" (színekhez) rendeli: mélyvíz, sekély víz, homok, fű, szikla és hó.

A Host program a Windows API (`GetAsyncKeyState`) segítségével valós időben figyeli a `W, A, S, D` billentyűk állapotát. Ezek az inputok egy `offset_x` és `offset_y` paramétert módosítanak, amelyeket minden képkockánál átadok a kernelnek. Mivel a Perlin-zaj matematikailag determinisztikus és végtelen, a koordináta-rendszer eltolásával a felhasználó egy végtelenített, előre nem legenerált kontinens felett repülhet át, nulla memória-előfoglalással (Zero Memory Footprint).

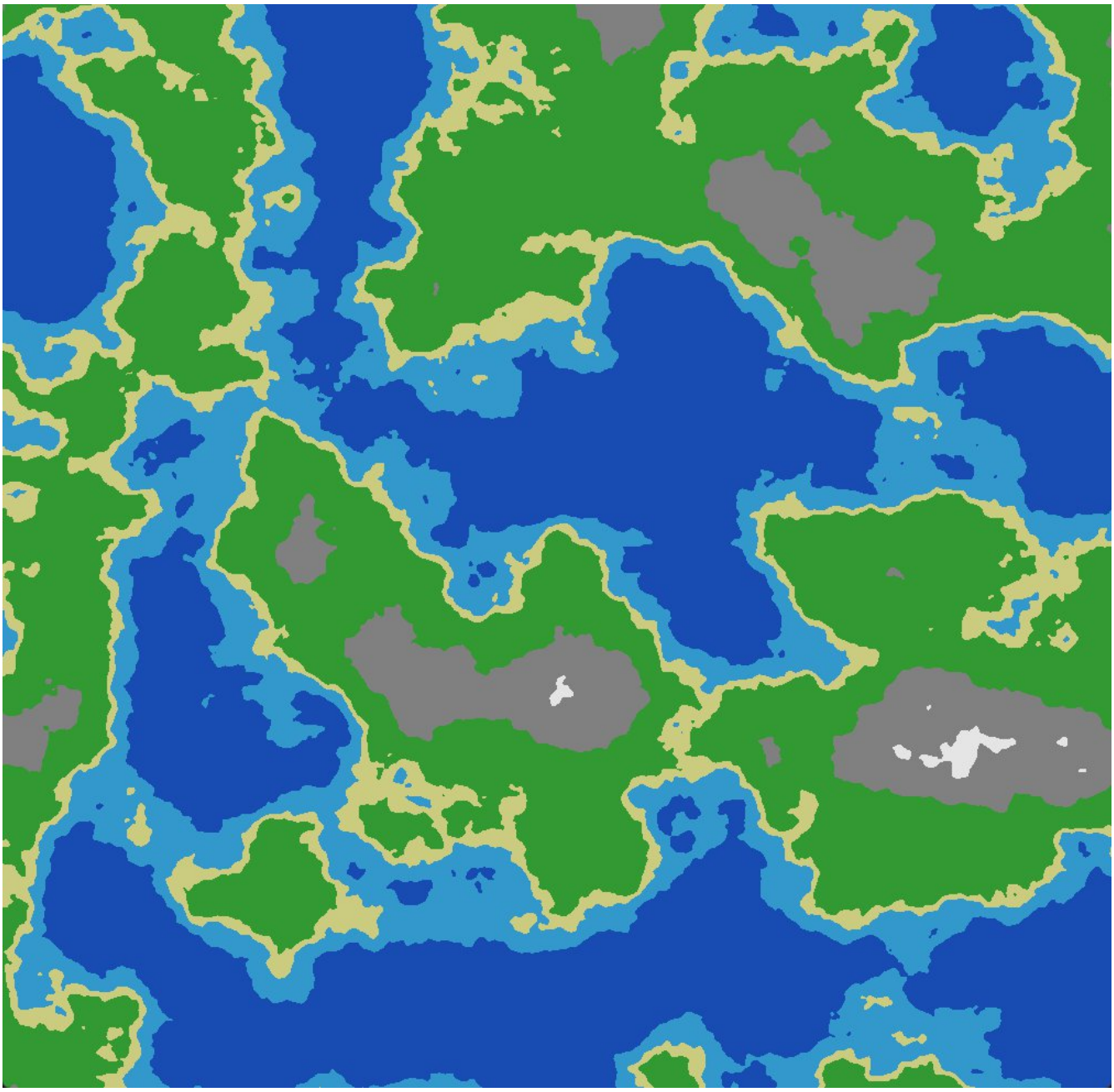

Figure 4: Interaktív procedurális terep